

Apuntes de Python

Apuntes introductorios a la programación procedimental en Python

Federico Melo Barrero
f.melo@uniandes.edu.co

Copyright © 2020, 2021 Federico Melo Barrero.

Todos los derechos reservados.

Ninguna parte de este documento, incluyendo el diseño de la portada, figuras e imágenes, puede ser reproducida, registrada, almacenada o transmitida en manera alguna ni por ningún medio, ya sea electrónico, eléctrico, fotoquímico, mecánico, electroóptico, magnético, de grabación, de fotocopia o cualquier otro sin permiso previo y por escrito del autor.

Índice

1. Programar	3
I Fundamentos	4
2. Valores, variables y expresiones	4
2.1. Valores	4
2.2. Variables	4
2.2.1. Sobreescritura y tipado dinámico	5
2.3. Expresiones	5
2.3.1. Operadores de cadenas de caracteres	5
2.3.2. Operadores aritméticos	6
2.3.3. Operadores de asignación compuesta	7
3. Funciones	7
3.1. Funciones de entrada y salida	8
3.1.1. Función print—	8
3.1.2. Función input—	8
II Condicionales	9
4. Booleanos	9
4.1. Operadores relacionales	9
4.1.1. Operadores de identidad	9
4.2. Operadores lógicos	9
4.2.1. Tablas de verdad	9
5. Estructura de control condicional	9
5.1. Cascadas de condicionales vs condicionales en secuencia	9
5.2. Expresión ternaria	9
6. Abordar problemas de programación con condicionales	9
7. Diccionarios	9
III Ciclos	10
8. Ciclo while	10
9. Ciclo for	11
10. ¿while o for?	11
10.1. De for a while	11

Preámbulo

Tomé estos apuntes cuando cursé Introducción a la Programación en la Universidad de los Andes, entre agosto y noviembre de 2020. Fueron refinados en mi tiempo como monitor en 2021, luego como tutor en el centro de enseñanza de programación de la universidad en 2022, y tras eso como asistente de investigación en ese mismo centro desde el 2022 hasta el 2024. Finalmente, fueron revisados en mi etapa como profesor de la asignatura entre el 2025 y 2026, pero no fueron modificados sustancialmente¹: siguen siendo apuntes de un estudiante para estudiantes.

1. Programar

Programar es, en esencia, escribir instrucciones que puedan ser ejecutadas por una máquina.

En general, escribir instrucciones para una máquina es más difícil que escribirlas para un humano por (al menos) tres razones:

- La máquina no entiende español, por lo que las instrucciones deben estar escritas en un lenguaje diseñado para que la máquina entienda. Existen centenas de dichos **lenguajes de programación**. **Python** es uno de ellos.
- Los lenguajes naturales (como el español) tienen reglas que determinan cómo organizar las palabras para que esté correctamente escrito. A eso se le llama **sintaxis**. Es fácil para un humano entender español ligeramente mal escrito, pero una máquina no puede interpretar Python con sintaxis incorrecta.
- El significado de un conjunto de palabras es su **semántica**. Los humanos pueden entender la intención detrás de instrucciones mal formuladas, pero una máquina ejecuta lo escrito de forma literal, así tenga poco sentido o no haga lo que se pretendía.

Esos tres factores hacen que programar sea difícil y que sea ampliamente considerado un arte.



Tip. En estos apuntes se usa mucho la palabra sintaxis. En esencia, es el orden en que se escriben las cosas, que es muy importante al programar.

¹De hecho, los apuntes están presentados en el mismo formato en el que originalmente fueron escritos; la clase de L^AT_EX que determina su estilo la escribí en noviembre del 2020.

Parte I

Fundamentos

2. Valores, variables y expresiones

2.1. Valores

Los textos en español se componen de palabras; los programas en Python se componen de **valores**. Algunos ejemplos de valores son números y secuencias de caracteres, como estos:

```
42
3.14
"Hola"
```

El primer valor es el número entero 42; el segundo, el número decimal 3.14; y el tercero, una secuencia de cuatro caracteres del teclado: H, o, l, a.

Análogo a como las palabras tienen categorías: sustantivos, verbos, etc.; los valores tienen **tipos de dato**²:

Tipo de dato	Nombre (inglés)	Traducción	Descripción	E.g.
int	<i>integer</i>	número entero	Números enteros	42, -17, 0, 1_000_000
float	<i>floating point number</i>	número de punto flotante	Números decimales	3.14, -0.5, .35
str	<i>string</i>	cadena de caracteres ³	Secuencias de caracteres del teclado	"Ho14", 'u78%6#a'

Cuadro 1: Algunos tipos de dato primitivos en Python

Las cadenas de caracteres deben escribirse entre comillas para que la máquina entienda dónde comienza y dónde termina la secuencia de caracteres que se está representando. Hay cuatro formas de escribirlas:

- Usando comillas dobles: `"Hola"`
- Usando comillas simples: `'H014'`
- Usando triples comillas dobles: `"""Hola"""`
- Usando triples comillas simples: `'''Python'''`

Si entre los caracteres de una cadena se quiere incluir algún tipo de comillas, es sensato delimitar la cadena con otro tipo de comillas, para que no haya ambigüedad. E.g.: `"Don't!"`, representa la secuencia de caracteres de teclado D, o, n, ', t, !. Si se hubiese escrito `'Don't!'`, no es claro cuál comilla termina la cadena.

Las triples comillas (dobles o simples) permiten representar cadenas de caracteres que contienen saltos de línea. E.g.:

```
"""Hola
Mundo"""
```

2.2. Variables

Una **variable** es un nombre que se asigna a un valor. Las variables se utilizan para almacenar valores en la memoria del programa.

Para definir una variable se utiliza el **operador de asignación**, que es el carácter `=`.

²Hay más tipos de dato en Python, que se exponen más adelante.

³La palabra *string* generalmente traduce a hilo y en algunos contextos a cadena. En el contexto de la programación, es convencional traducirla como cadena de caracteres.



Tip. El símbolo de igualdad en matemáticas y el operador de asignación en Python utilizan el mismo carácter: `=`. Sin embargo, en cada contexto significa cosas diferentes.

En programación sucede bastante que se utilizan caracteres que en algún otro contexto significan otra cosa.

La sintaxis, de izquierda a derecha, se compone de: el nombre de la variable, el operador de asignación y el valor que se le asigna a la variable. E.g.:

```
persona = "David"
altura = 1.77
```

Allí:

- Se toma la secuencia de caracteres `D`, `a`, `v`, `i`, `d` y se almacena en la memoria del programa bajo el identificador `persona`.
- Se toma el número decimal `1.77` y se almacena en la variable `altura`.

Tiene sentido otorgar nombres a los valores para poder usarlos después. Los identificadores de las variables deben seguir las siguientes reglas:

- Deben ser alfanuméricos: letras, números y guiones bajos.
- No pueden contener espacios; para usar varias palabras dentro del identificador, se separan con guiones bajos: `nombre_de_la_variable`.
- No pueden empezar por dígitos ni deben empezar por mayúsculas.
- No pueden ser **palabras reservadas** del lenguaje, que son palabras específicas que ya tienen un significado en Python.⁴

2.2.1. Sobreescritura y tipado dinámico

Se puede sobrecribir el valor de una variable por otro y la máquina olvida el valor anterior. E.g.:

```
persona = 3
```

Ahora, la variable `persona` tiene el valor `3` y deja de hacer referencia el valor anterior, `"David"`.

Además, cambió el tipo de dato de la variable `persona`, que antes era `str` y ahora es `int`. Cambiar el tipo de dato de una variable es posible en Python, pero no lo es en muchos otros lenguajes de programación. A esa característica de Python se le llama **tipado dinámico**⁵.

2.3. Expresiones

Una **expresión** es una combinación de valores y operadores que el intérprete de Python puede evaluar para obtener un valor. En matemática, se puede evaluar la expresión $1 + 1$ para obtener el valor `2`; en Python, es similar.

Los **operadores** son símbolos que representan operaciones entre valores de un tipo de dato determinado.

2.3.1. Operadores de cadenas de caracteres

Las operaciones que se pueden realizar con valores `str` son la concatenación y la repetición:

⁴Usualmente en este punto los libros de texto listan las palabras reservadas. Yo no lo hago porque, primero, no es necesario memorizarlas; y, segundo, es bastante improbable que el nombre de una variable preciso coincida con una palabra reservada.

⁵Lo descrito en este párrafo es realmente una consecuencia del tipado dinámico, no una definición. Una definición escapa el alcance de estos apuntes, pero podría ser: un lenguaje es dinámicamente tipado si su sistema de tipos realiza verificaciones en tiempo de ejecución, en lugar de en tiempo de compilación.

Operador	Símbolo	Descripción	E.g.	Resultado
Concatenación	+	Une dos cadenas de caracteres	"Hola" + "Mundo"	"HolaMundo"
Repetición	*	Opera un <code>str</code> con un <code>int</code> ; repite el <code>str</code> tantas veces como indica el <code>int</code>	"Hola" * 3	"HolaHolaHola"

Cuadro 2: Operadores de cadenas de caracteres en Python

2.3.2. Operadores aritméticos

Las operaciones que se pueden realizar con valores de tipo de dato numérico, i.e. `int` o `float`, son las operaciones aritméticas convencionales:

Operador	Símbolo	Ejemplo matemático	Ejemplo en Python	Resultado
Exponenciación	**	3^4	<code>3**4</code>	81
Multiplicación	*	7×6	<code>7 * 6</code>	42
División	/	$11/4$	<code>11 / 4</code>	2.75
División entera	//	$\lfloor 11/4 \rfloor$	<code>11 // 4</code>	2
Módulo	%	$11 \bmod 4$	<code>11 % 4</code>	3
Suma	+	$10 + 7$	<code>10 + 7</code>	17
Resta	-	$20 - 8$	<code>20 - 8</code>	12

Cuadro 3: Operadores aritméticos en Python

Las operaciones se pueden combinar para construir expresiones más complejas. E.g.: $2 + 4 * 2$ evalúa a 10. Python no evalúa ingenuamente de izquierda a derecha, sino que respeta la **jerarquía de operaciones**⁶ clásica de la aritmética.

División entera y módulo. Seguramente el único par de operadores no familiares en la tabla 3 son los operadores de división entera (`//`) y módulo (`%`).

Cuando se realiza una división, las partes son:

$$\text{dividendo} / \text{divisor} = \text{cociente}$$

Usualmente el cociente se expresa como un número decimal, pero también se puede entender como un número entero y un residuo:

$$\text{dividendo} = \text{divisor} \times \text{cociente entero} + \text{residuo}$$

La división entera representa el cociente entero, y el módulo representa el residuo:

$$\text{dividendo} // \text{divisor} = \text{cociente entero}$$

$$\text{dividendo} \% \text{divisor} = \text{residuo}$$

E.g., $11/4$ es igual a 2,75, pero también se puede expresar como $11 = 4 \times 2 + 3$. En ese caso, la división entera de 11 entre 4 es 2, que es simplemente tomar la parte entera del cociente⁷, y el módulo de 11 entre 4 es 3.

Más intuitivamente, la división entera representa la cantidad de veces que el un número (el divisor) cabe completamente dentro de otro (el dividendo), y el módulo representa lo que sobra. Si se reparte una torta de 11 pedazos equitativamente entre 4 personas, a cada una le tocan 2 pedazos (el cociente entero) y sobran 3 pedazos que no le tocan a nadie (el módulo).

⁶Estos apuntes presuponen familiaridad con la jerarquía de operaciones; sin embargo, en esencia, el orden es: 1) exponenciación; 2) multiplicación y división (incluyendo división entera y módulo); y 3) suma y resta. Las operaciones con igual prioridad se evalúan de izquierda a derecha. Si existen paréntesis (o corchetes), las operaciones dentro de ellos tienen prioridad sobre las externas, y los paréntesis más internos se evalúan antes que los más externos.

⁷Nótese que esto no es lo mismo que aproximar.

2.3.3. Operadores de asignación compuesta

Es común en programación modificar el valor de una variable operándola con algún valor. E.g.,

```
x = 10
x = x + 5
```

Que actualiza el valor de `x` a 15. Existe una sintaxis más corta para escribir este tipo de operaciones, que es la siguiente:

```
x = 10
x += 5
```

El operador `+=` es un operador de asignación compuesta. Existe uno para cada operador aritmético y de cadenas de caracteres:

Operador	Ejemplo	Asignación equivalente
<code>+=</code>	<code>x += 5</code>	<code>x = x + 5</code>
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>
<code>*=</code>	<code>x *= 2</code>	<code>x = x * 2</code>
<code>/=</code>	<code>x /= 4</code>	<code>x = x / 4</code>
<code>//=</code>	<code>x //= 3</code>	<code>x = x // 3</code>
<code>%=</code>	<code>x %= 2</code>	<code>x = x % 2</code>
<code>**=</code>	<code>x **= 3</code>	<code>x = x ** 3</code>
<code>+=</code>	<code>s += " Mundo"</code>	<code>s = s + " Mundo"</code>
<code>*=</code>	<code>s *= 3</code>	<code>s = s * 3</code>

Cuadro 4: Operadores de asignación con operador en Python

A estas alturas, sabemos:

- Qué es un valor
- Algunos tipos de dato primitivos en Python
- Qué es una variable y para qué sirve el operador de asignación.
- Qué es una expresión
- Cuáles operaciones pueden aplicarse sobre los tipos de dato vistos.
- Cómo funcionan los operadores de asignación compuesta

Eso no permite hacer programas muy complejos, pero sí nos permite escribir programas que hagan cálculos y almacenen resultados. Ya en este punto, Python es más poderoso que la mayoría de calculadoras avanzadas. Probablemente, soporta cálculos más extensos que su calculadora.

3. Funciones

Similar a como las variables son nombres que se asignan a valores, las funciones son nombres que se asignan a procedimientos. Esto permite pensar y escribir un pedazo de código sola una vez y luego reutilizar esa misma lógica múltiples veces.

Distinguimos entre dos tipos de funciones: las funciones *built-in* (integradas o nativas), que son funciones que creó alguien más y ya vienen predefinidas en Python, y las funciones *user-defined* (definidas por el usuario), que son aquellas que creamos nosotros.

3.1. Funciones built-in

3.1.1. Funciones de conversión de tipos

3.1.2. Algunas funciones matemáticas

3.1.3. Funciones de entrada y salida

Las funciones de entrada y salida se utilizan en el programa para interactuar con el usuario.

La función `print` (que traduce a imprima) recibe un argumento y lo muestra en la consola.

La función `input` se utiliza para leer un valor de la consola. Es decir, se utiliza para que el usuario ingrese un valor y se lo asigne a una variable.

3.1.4. ??

3.2. Definición de funciones

Parte II

Condicionales

4. Booleanos

4.1. Operadores relacionales

4.1.1. Operadores de identidad

4.2. Operadores lógicos

4.2.1. Tablas de verdad

5. Estructura de control condicional

La estructura de control condicional en Python utiliza la palabra `if`, que traduce al español como *si*. Su sintaxis es: `if valor de tipo bool:`

`bloque de código` Por ejemplo:

```
if True:
    print("Hello, World!")
```

El bloque de código se ejecuta si la condición es verdadera.

5.1. Cascadas de condicionales vs condicionales en secuencia

5.2. Expresión ternaria

6. Abordar problemas de programación con condicionales

7. Dicionarios

Parte III

Ciclos

Python tiene dos tipos de ciclos: `while` y `for`. Un **ciclo** es una estructura de control que permite repetir un bloque de código varias veces.

8. Ciclo `while`

El ciclo `while` ejecuta un bloque de código mientras una condición sea verdadera. Su sintaxis es análoga a la sintaxis del `if`:

```
while valor de tipo bool :
    bloque de código
```

La diferencia es: en el `if`, el bloque de código se ejecuta cuando la condición es verdadera y se ejecuta una sola vez; en el `while`, el bloque de código se ejecuta una y otra vez mientras la condición siga siendo verdadera.

Dicho de otra forma, el bloque de código solo se deja de ejecutar cuando la condición sea falsa. Eso implica que diseñar un ciclo `while` consiste básicamente de:

- Una condición que inicialmente sea verdadera (si no, el ciclo nunca ejecutará).
- El bloque de código que se ejecutará repetidamente.
 - Dentro del bloque de código, debe haber algo que permita que el valor de la condición cambie. De lo contrario, el bloque de código se seguirá ejecutando indefinidamente. A ese tipo de error se le llama un *ciclo infinito*.

Un ejemplo de ciclo `while` es el siguiente:

```
i = 0
while i < 3:
    print(i)
    i += 1
```

En ese ejemplo:

- La condición es `i < 3`, que es verdadera inicialmente.
- El bloque de código imprime el valor de `i` y luego incrementa el valor de `i` en 1.
 - Como siempre se incrementa el valor de `i`, eventualmente `i` valdrá 3 y la condición será falsa. En ese punto, el bloque de código se deja de ejecutar.

Con lo anterior en mente, resulta evidente que el ciclo se ejecuta 3 veces:

1. La primera ejecución se debe a que `i` vale 0 y 0 es menor que 3. Se imprime el valor 0 y luego se incrementa `i` en 1. Al final de esa ejecución, `i` vale 1.
2. Luego, se revisa si se cumple la condición. En este caso, sí se cumple, porque 1 es menor que 3. Se imprime 1, y se incrementa, de forma que `i` ahora vale 2.
3. En la tercera iteración, se sigue cumpliendo la condición (2 es menor que 3); se imprime 2 y ahora `i` vale 3.
4. Tras eso, se revisa si se cumple la condición. Ahora, ya no se cumple: 3 no es menor que 3. El bloque de código se deja de ejecutar.

9. Ciclo for

El ciclo `for` ejecuta un bloque de código para cada elemento de un iterable. Un **iterable** es cualquier objeto que puede recorrerse elemento por elemento. Conocemos tres tipos de datos que son iterables: cadenas de caracteres, listas y diccionarios.

Un ejemplo de ciclo `for` es el siguiente:

```
for char in "Hola":
    print(char)
```

En ese ejemplo:

- El iterable es la cadena de caracteres "Hola". Los elementos que se recorren son: H, o, l, a.
- La variable `char` representa un elemento distinto en cada iteración.
- El bloque de código imprime el valor de `char` en cada iteración.

Con lo anterior en mente, resulta evidente que el ciclo se ejecuta 4 veces: en la primera iteración, `char` vale H y se imprime H; en la segunda `char` vale o y se imprime o; en la tercera, se imprime l; y en la cuarta, a.

La sintaxis general es:

```
for variable de iteración in iterable:
    bloque de código
```

10. ¿while o for?

Técnicamente, los ciclos `while` y `for` son equivalentes. Es decir, ambos pueden resolver los mismos problemas igual de bien. Sin embargo, por algo existen ambos: no tiene sentido usarlos para lo mismo.

10.1. De for a while

La conversión radica en recorrer el iterable que recorrería el ciclo `for` pero mediante un ciclo `while`. Para eso, el ciclo `while` simplemente incrementa un contador y se usa ese contador para acceder a los elementos del iterable. Es decir:

```
i = 0
while i < len(iterable):
    variable de iteración = iterable[i]
    bloque de código
    i += 1
```